94.3
Pass

Hemasundhar

PDF generated at: 5 Mar 2026 13:43:31 UTC

View this report on HackerRank

Score

94.3% • 165.0 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 80 min 59 sec on 5 Mar 2026 17:50:24 IST

Candidate Information

Email	hemasundharpv@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	5 Mar 2026 17:50:24 IST
Time taken	80 min 59 sec/ 90 min
Work Experience	4 years
Invited by	Priyanshi
Invited on	5 Mar 2026 15:25:51 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	33%

2	Spring Boot Intermediate	100%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	60%	Hard	33%
TDD	0%	Medium	100%
Exception Handling	100%	Aspect-Oriented Programming	100%
Implementation	100%	Arrays	100%
Algorithms	100%	Problem Solving	100%
Greedy	100%	Java	100%

Questions

Spring Boot (Advanced) • 15 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Cracker Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min 53 sec	✓

2	Determine the Output Multiple Choice	Spring Boot (Advanced)	0/5	-	1 min	✗
3	Output Multiple Choice	Spring Boot (Advanced)	0/5	-	42 sec	✗
4	Endpoint Security Multiple Choice	Spring Boot (Intermediate)	5/5	-	1 min 4 sec	✓
5	Predict the Output Multiple Choice	Spring Boot (Intermediate)	5/5	-	54 sec	✓

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Wind Farms Coding	Problem Solving (Intermediate)	75/75	-	54 min 25 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status

7	Java: Optimized Sequential Video Watching Coding	Java (Intermediate)	75/75	-	20 min 34 sec	✓

1. Cracker

Correct

Multiple Choice | Spring Boot | Hard

Question description

Cracker.java

```
@Component
public class Cracker {
    public void startCracking(){
        System.out.println("Cracking...");
    }
    public void stopCracking(){
        System.out.println("Cracking Stopped!");
    }
}
```

CrackerService.java

```
@Service
public class CrackerService {

    @Autowired
    Cracker cracker;
    public void workWithNewCracker() {
        cracker.stopCracking();
        cracker = new Cracker();
    }
    public void startWorking() {
        cracker.startCracking();
    }
}
```

CrackerWorker.java

```
@Service
public class CrackerWorker {

    @Autowired
    CrackerService crackerService;

    public void startWorking() {
        crackerService.startWorking();
        crackerService.workWithNewCracker();
        crackerService = new CrackerService();
        crackerService.startWorking();
    }
}
```

Consider these classes are managed in Spring Boot. What happens when 'crackerWorker.startWorking()' is called in the Spring context?

Interviewer guidelines

CrackerService in CrackerWorker will work perfectly until it's replaced with new CrackerService. When that happens it just replace existing injected bean with empty pojo. Which doesn't have it's cracker object so when it's startWorking method is called it will try to execute null cracker attributes startWorking method and NullPointerException will arise.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- "Cracking..." "Cracking Stopped!" will be written to console

- "Cracking..." "Cracking Stopped!" "Cracking..." will be written to console

"Cracking..." "Cracking Stopped!" NullPointerException will be thrown

None of above

No comments.

2. Determine the Output

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
   public static void main(String[] args) {
      SpringApplication.run(SpringQuestionsApplication.class, args);
   }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
   @Override
   public void run(String... args) throws Exception {
      System.out.println("Truth is behind the lies");
```

```
        System.out.println("This is working perfectly");
        throw new Exception("Does this work?");
    }
}
```

Truth.java

```
package com.hackerrank.spring.reality;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
    @Override
    public void run(String... args) throws Exception {
        System.out.println("Truth is in your eyes");
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

Interviewer guidelines

Correct Answer:

Option [A] – "Truth is in your eyes"

Explanation: Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

Incorrect Answers:

Option [B] – Multiple prints + exceptions

Explanation: `Lie` never runs, so its prints and exception do not occur.

Option [C] – Prints from Lie only

Explanation: `Truth` is discovered and executed; `Lie` is not.

Option [D] – Prints from Lie, then Truth

Explanation: `Lie` is outside the scan path and never executed.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Truth is in your eyes
- Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops
- Truth is behind the lies This is working perfectly And exception occurs and application stops
- **Truth is behind the lies This is working perfectly And exception occurs containing "Does this work?" Truth is in your eyes**

No comments.

3. Output

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
```

```
    SomeClassToTest someClassToTest;

    @Before
    public void init()
    {
      log = new File("log.txt");
      logWriter = new FileWriter(log);
    }

    @After
    public void teardown()
    {
       log.delete();
    }

    @Test
    public void test1()
    {
       logWriter.write("Entered Test 1");
       assertTrue(someClassToTest.someMethod());
    }

    @Test
    public void test2()
    {
       logWriter.write("Entered Test 2");
       assertTrue(someClassToTest.otherMethod());
    }
  }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The output file is not created.
- ● **The output file is created but not deleted.**

- The output file is created and eventually deleted. ✓
- The create and delete blocks will not be executed.

No comments.

4. Endpoint Security

Correct

Multiple Choice | Spring Boot | Medium

Question description

In Spring Boot Security, the option to enable secure endpoints with @PreAuthorize annotation is:

Interviewer guidelines

@EnableGlobalMethodSecurity by default doesn't enable this feature. To enable this prePostEnabled=true value must be set

Candidate's Solution

Options: (Expected answer indicated with a tick)

- Configuration class annotated with @EnableGlobalMethodSecurity
- Configuration class annotated with @EnableGlobalMethodSecurity(securedEnabled = true)

Configuration class annotated with @EnableGlobalMethodSecurity(prePostEnabled = true)

None above

No comments.

5. Predict the Output

Correct

Multiple Choice | Medium | Exception Handling | Spring Boot | Aspect-Oriented Programming

Question description

Consider the following Snippet and predict the output.

The following API is triggered: http://localhost:8080/add/employee?eId=123&fName=Ravi&lastName=Shekhar

```
@Autowired
private EService eService;

@RequestMapping(value = "/add/employee", method = RequestMethod.GET)
public com.model.Employee add(@RequestParam("eId") String eId, @RequestParam("fName") Strin
g fName, @RequestParam("lastName") String lastName)
{
   return eService.createEmployee(eId, fName, lastName);
}

@Aspect
@Component
public class EServiceAspect
{
   @After(value = "execution(* com.random.service.EService.*(..)) && args(eId, fName, lastName)")
   public void afterAdvice(JoinPoint joinPoint, String eId, String fName, String lastName) {
```

```
        System.out.println("After method:" + joinPoint.getSignature());
        System.out.println("Creating Employee with first name - " + fName+ ", second name - " + lastNa
me+ " and id - " + eId);
    }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] ** After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Ravi , second name - Shekhar and id - 123   **

- [ ] After method : com.test.model.Employee addEmployee(string,string,string) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(empId,firstName,secondName) Creating Employee with first name - Ravi , second name - Shekhar and id - 123

- [ ] After method : com.test.model.Employee addEmployee(String,String,String) Creating Employee with first name - Shekhar, second name - Ravi and id - 123

No comments.

6. Wind Farms

Correct

Coding | Implementation | Medium | Arrays | Algorithms | Problem Solving

Question description

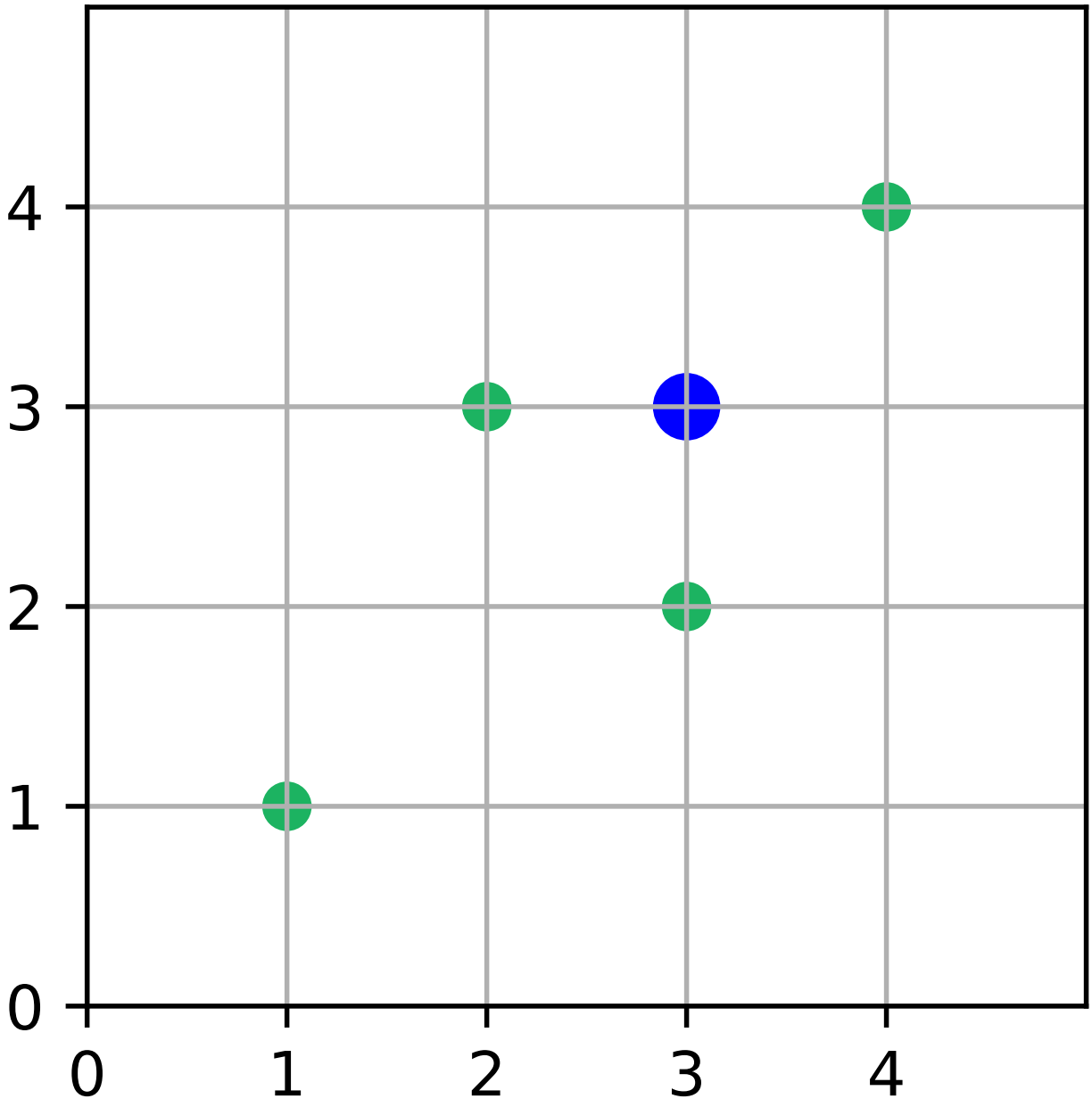
An energy company needs to place a control center to manage a distributed wind farm. Your task is to determine the optimal location for the control center that minimizes the total connection cost.

For each wind farm *i*:

- The farm is located at coordinates *(x[i], y[i])*
- The control center is at coordinates *(cx, cy)*
- The distance is calculated as *|x[i] - cx| + |y[i] - cy|*
- The connection cost is *premium[i] * distance*

Example

x = [1, 3, 2, 4]
y = [1, 2, 3, 4]
premium = [1, 3, 2, 4]



The optimal location is at position [3, 3], resulting in:

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 3| = 4	1	4 * 1 = 4

1	(3, 2)	|3 - 3| + |2 - 3| = 1	3	1 * 3 = 3
2	(2, 3)	|2 - 3| + |3 - 3| = 1	2	1 * 2 = 2
3	(4, 4)	|4 - 3| + |4 - 3| = 2	4	2 * 4 = 8

The total connection cost is 17, which is the minimum possible.

Function Description

Complete the function *windFarms* in the editor with the following parameter(s):

int premium[n]: the distance multiplier for the cost to connect
int x[n]: *x* coordinates of the wind farms
int y[n]: *y* coordinates of the wind farms

Returns

int: the minimum cost to connect the wind farms

Constraints

- $1 \le n \le 10^3$
- $1 \le x[i], y[i] \le 10^4$
- $1 \le premium[i] \le 50$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer *n*, the size of the array *premium*.
The next *n* lines each contain an element, *premium[i],* where 0 ≤ i < n.
The next line contains an integer *n*, the size of *x[]*.
The next *n* lines each contain an element, *x[i]*.
The next line contains an integer *n*, the size of *y[]*.
The next *n* lines each contain an element, *y[i]*.

Note: *n* is the same value in each instance.

▼ SAMPLE CASE 0

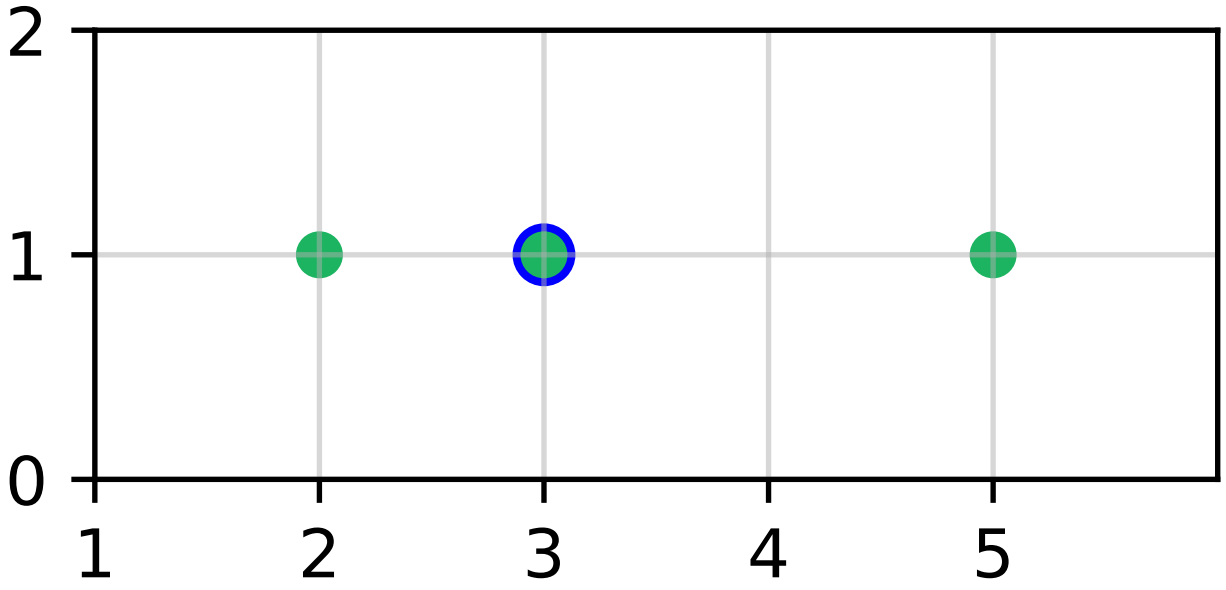
Sample Input 0

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
2    →  x = [2, 3, 5]
3
5
3    →  y[] size n = 3
1    →  y = [1, 1, 1]
1
1
```

Sample Output 0

```
3
```

Explanation 0



Locate the controller at farm 1.

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 1| = 1	1	1 * 1 = 1
1	(3, 1)	|3 - 3| + |1 - 1| = 0	1	0 * 1 = 0

2	(5, 1)	|5 - 3| + |1 - 1| = 2	1	2 * 1 = 2

▼ SAMPLE CASE 1

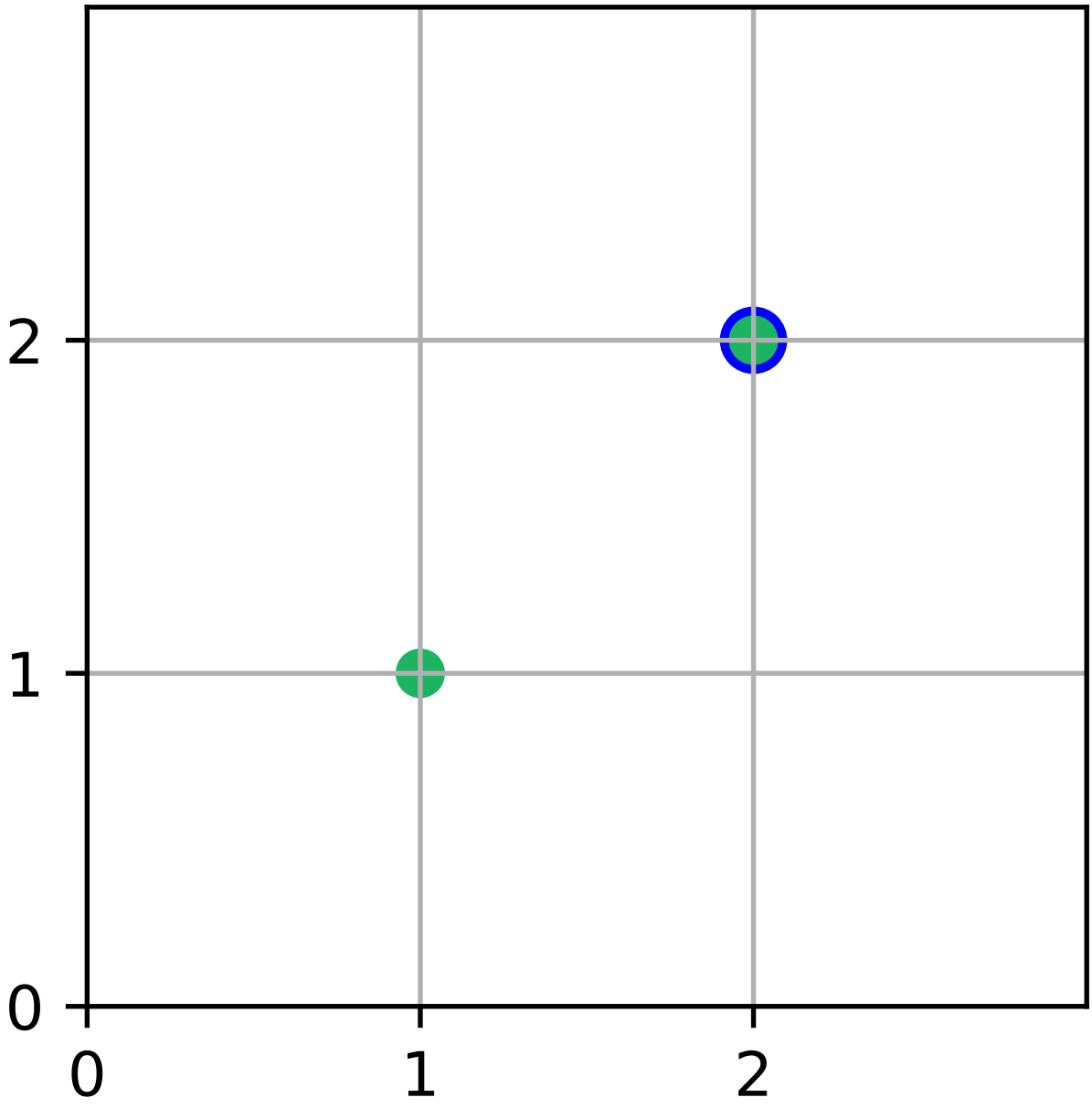
Sample Input 1

```
STDIN    Function
-----    --------
2    →  premium[] size n = 2
1    →  premium = [1, 10]
10
2    →  x[] size n = 2
1    →  x = [1, 2]
2
2    →  y[] size n = 2
1    →  y = [1, 2]
2
```

Sample Output 1

```
2
```

Explanation 1



The sum of the distances to a center at (1, 1), (1, 2), (2, 2), or (2, 1) is 2. Since the premium to connect farm 1 is 10, it is best to locate at position (2, 2) for a cost of 2 * 1 + 0 * 10 = 2.

▼ SAMPLE CASE 2

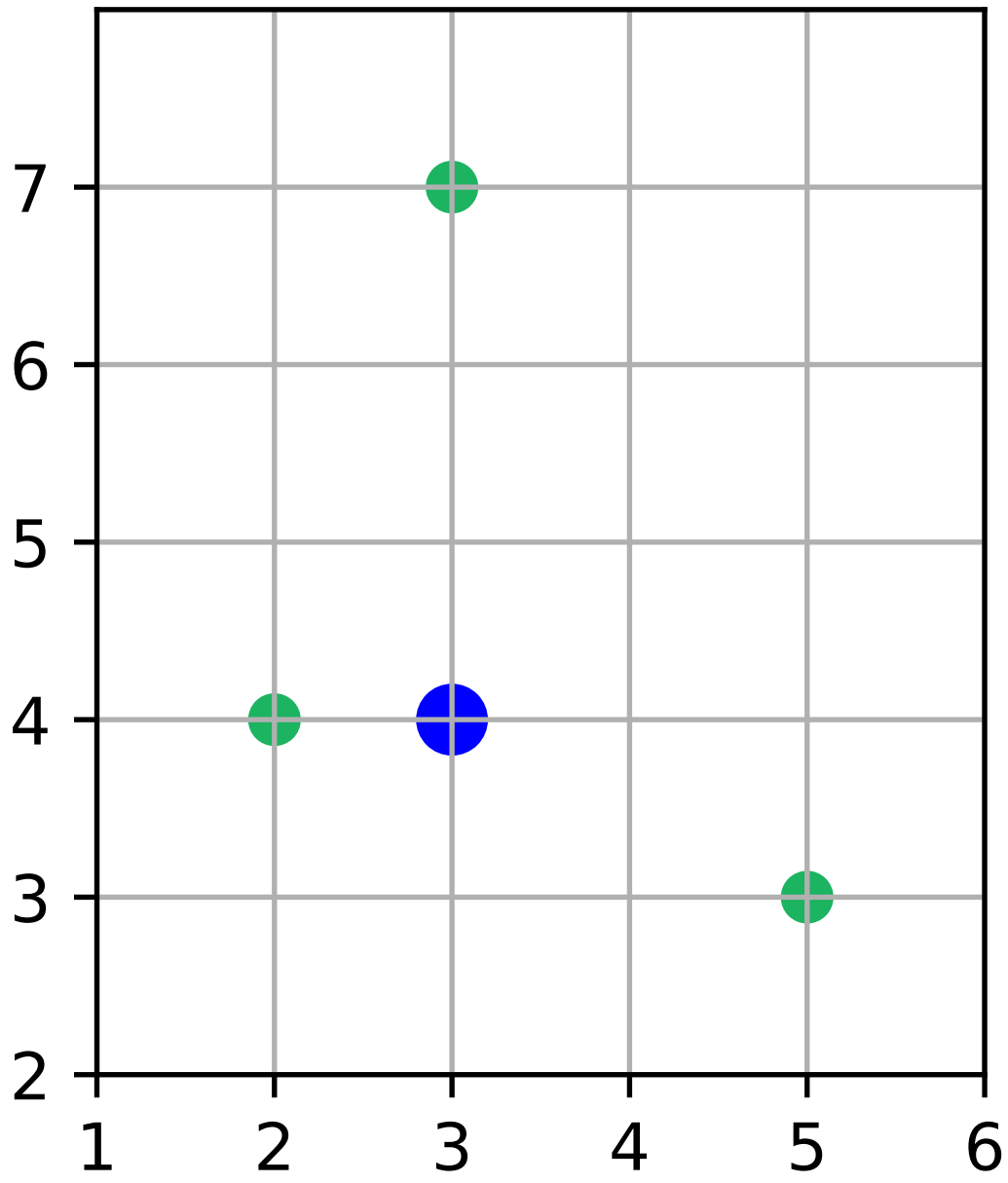
Sample Input 2

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
5    →  x = [5, 2, 3]
2
3
3    →  y[] size n = 3
3    →  y = [3, 4, 7]
4
7
```

Sample Output 2

```
7
```

Explanation 2



The optimal location is (3, 4) for a cost of 3 + 1 + 3 = 7.

Interviewer guidelines

▼ SOLUTION

Skills: Loops and counters, I/O

The cost can be calculated for x and y coordinates separately. The function examines each possible x and y coordinate for the control center separately (from 1 to 10000), computes the total cost of connecting all farms for each possible coordinate, and captures the minimum cost found. The ultimate minimum cost is the summation of these minimum costs for x and y coordinates separately.

Optimal Solution

```
def windFarms(premium, x, y):
    minCostX = 10**10
    n = len(x)

    for cx in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cx - x[i])

        minCostX = min(minCostX, cost)

    minCostY = 10**10
    for cy in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cy - y[i])

        minCostY = min(minCostY, cost)

    return minCostX + minCostY
```

▼ **COMPLEXITY ANALYSIS**

Time Complexity - O(n)
The universe of x and y values are tested against 10000 values.

Space Complexity - O(1)
Constant space is required regardless of n.

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'windFarms' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY premium
     *  2. INTEGER_ARRAY x
     *  3. INTEGER_ARRAY y
     */

    public static int windFarms(List<Integer> premium, List<Integer> x, List<Integer> y) {
    // Write your code here

    int n = premium.size();
    Integer [] idxX = new Integer[n];
    for (int i =0; i < n; i++) idxX[i] = i;
    Arrays.sort(idxX, (a,b) ->x.get(a) - x.get(b));
    long totalWeight = 0;
    for (int w : premium) totalWeight += w;
    long cumW = 0;
    int optX =  x.get(idxX[n-1]);
    for(int i =0; i< n; i++){
        cumW += premium.get(idxX[i]);
        if(cumW *2 >= totalWeight){
            optX = x.get(idxX[i]); break;
        }
    }
```

```java

    Integer[] idxY  =  new Integer[n];
    for(int i =0; i <n ; i++) idxY[i] = i;
    Arrays.sort(idxY, (a,b) -> y.get(a) - y.get(b));

    cumW = 0;
    int optY = y.get(idxY[n-1]);

    for(int i =0; i < n; i++){
        cumW += premium.get(idxY[i]);
        if(cumW *2 >= totalWeight) {
            optY = y.get(idxY[i]);
            break;
        }
    }

    int totalCost = 0;
    for(int i =0 ; i< n; i++){
        totalCost += premium.get(i) * (Math.abs(x.get(i) - optX)+ Math.abs(y.get(i) - optY));
    }

    return totalCost;

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int premiumCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> premium = IntStream.range(0, premiumCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
```

```
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int xCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> x = IntStream.range(0, xCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int yCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> y = IntStream.range(0, yCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.windFarms(premium, x, y);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED

TestCase 0	Easy	Sample	Success	1	0.0494 sec	37.3 KB
TestCase 1	Easy	Sample	Success	1	0.0799 sec	37.9 KB
TestCase 2	Easy	Sample	Success	1	0.0502 sec	37.7 KB
TestCase 3	Easy	Sample	Success	2	0.0501 sec	37.9 KB
TestCase 4	Easy	Hidden	Success	2	0.0782 sec	38 KB
TestCase 5	Easy	Hidden	Success	2	0.0541 sec	37.6 KB
TestCase 6	Medium	Sample	Success	6	0.0788 sec	38.3 KB
TestCase 7	Medium	Hidden	Success	6	0.061 sec	38.2 KB
TestCase 8	Medium	Hidden	Success	6	0.0545 sec	38.1 KB
TestCase 9	Hard	Hidden	Success	12	0.0902 sec	43.5 KB
TestCase 10	Hard	Hidden	Success	12	0.0928 sec	43.9 KB
TestCase 11	Hard	Hidden	Success	12	0.0996 sec	43.5 KB
TestCase 12	Hard	Hidden	Success	12	0.1114 sec	43.6 KB

No comments.

7. Java: Optimized Sequential Video Watching

Correct

Coding Medium Greedy Java

Question description

A learner is given a sequential playlist of *n* videos to enhance their knowledge. Implement a function to calculate the minimum time required to complete a total of *m* viewings while adhering to the platform's constraints.

System Constraints

- Sequential Viewing:
  - The learner must watch videos in the order they appear in the playlist.
  - To watch a video at index *i*, the learner must first watch all videos from index 0 to *i-1* at least once.
- Initial Full Viewing:
  - To view any video at index *i* for the first time, the learner must spend *firstWatch[i] + repeatWatch[i]* minutes. This ensures a complete understanding of the video.
- Subsequent Viewings:
  - For subsequent viewings of a video at index *i*, the learner spends only *repeatWatch[i]* minutes.
  - At any time, they can rewatch any video they have already watched.
- Minimum Total Watch Count:
  - The learner must achieve at least *m* total viewings across all videos.
  - Videos can be rewatched as desired, and not all videos need to be watched equally.

Return the minimum time required to complete *m* viewings of videos while following these constraints.

Example

If *n* = 4, *m* = 4, *firstWatch* = [1, 5, 9, 11], and *repeatWatch* = [2, 7, 10, 11], the optimal strategy is as follows:

- The learner watches the 0th video for the first time, spending *firstWatch*[0] + *repeatWatch*[0] = 1 + 2 = 3 minutes.
- The learner then rewatches the 0th video 3 more times in, spending *repeatWatch*[0]*3 = 2 * 3 = 6 minutes.
- The learner has successfully completed 4 viewings. Thus, the total time required is 3 + 6 = 9 minutes, and 9 is returned.

Function Description

Complete the function *findMinMinutes* in the editor below.

Function Parameters

int m: the total number of video viewings to complete

int firstWatch[n]: the minutes required to watch each video the first time

int repeatWatch[n]: the minutes required to rewatch each video

Returns

long: the minimum minutes required for the learner to reach their target number of viewings

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 10^9$
- $1 \le firstWatch[i], repeatWatch[i] \le 10^9$

INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *m*.

The next line contains an integer, *n,* the size of *firstWatch*.

Each of the next *n* lines contains an integer, *firstWatch[i]*.

The next line contains an integer, *n,* the size of *repeatWatch*.

Each of the next *n* lines contains an integer, *repeatWatch[i]*.

SAMPLE CASE 0

Sample Input 0

```
STDIN    FUNCTION
-----    --------
5        m = 5
3        firstWatch[] size n = 3
6        firstWatch = [6, 8, 9]
```

```
8
9
3       repeatWatch[] size n = 3
4       repeatWatch = [4, 2, 10]
2
10
```

Sample Output 0

```
26
```

Explanation

The goal is to complete 5 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time, so the time required is *firstWatch*[0] + *repeatWatch*[0] = 6 + 4 = 10 minutes.
- The learner watches the 1st video for the first time, so the time required is *firstWatch*[1] + *repeatWatch*[1] = 8 + 2 = 10 minutes.
- The learner then rewatches the 1st video 3 times. So the time taken = *repeatWatch*[1]*3 = 2 * 3 = 6 minutes.
- The total time required to complete 5 video viewings is 10 + 10 + 6 = 26 minutes.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
6         m = 6
5         firstWatch[] size n = 5
2         firstWatch = [2, 3, 7, 8, 4]
3
7
8
4
5         repeatWatch[] size n = 5
3         repeatWatch = [3, 6, 8, 7, 6]
6
8
7
6
```

Sample Output 1

```
20
```

Explanation

The goal is to complete 6 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time in *firstWatch*[0] + *repeatWatch[0]* = 2 + 3 = 5 minutes.
- The learner then rewatches the 0th video 5 times in *repeatWatch[0]**5 = 3 * 5 = 15 minutes.
- The total time required to complete 6 viewings is 5 + 15 = 20 minutes.

Interviewer guidelines

▼ SOLUTION

Description

The greedy solution suggests that, at any given index, if the time to watch the video is less than at any other index between i+1 and n, the minimum total time will be the sum of the watch time up to index i-1, plus the repeat time of the ith video multiplied by (m-i).

Solution

```
class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
        long minMinutes = Long.MAX_VALUE;
        long cur = 0;
        int n = firstWatch.size();

        for (int i = 0; i < Math.min(n,m); i++) {
            int a = firstWatch.get(i);
            int b = repeatWatch.get(i);

            minMinutes = Math.min(minMinutes, cur + ((long) (m - i) * b) + a);

            cur += a + b;
        }
        return minMinutes;
```

```
    }

}
```

▼ COMPLEXITY

Time Complexity - O(n)

The algorithm iterates through each number in *firstWatch* and *repeatWatch* exactly once, performing constant-time checks and arithmetic operations in each iteration. Thus, the time complexity is O(n), where *n* is the length of *firstWatch*.

Space Complexity - O(1)

Space usage is constant, only utilizing a fixed number of variables (*mn, cur,* etc.) that do not scale with the length of *firstWatch*. Therefore, the space complexity is O(1).

Candidate's Solution

Language used: **Java 17**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

```

```
    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {

        int n =  firstWatch.size();
    long totalTime =  0;


    long minRepeat =  Long.MAX_VALUE;

    long best  =  Long.MAX_VALUE;

    for(int k=1 ; k <= Math.min(n,m); k++){

        totalTime += firstWatch.get(k-1) + repeatWatch.get(k-1);

        minRepeat =  Math.min(minRepeat,  repeatWatch.get(k-1));

        long remaining = m-k;

        long cost =  totalTime + remaining * minRepeat;
        best  = Math.min(best,cost);
    }

    return best;

    }

}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int m = Integer.parseInt(bufferedReader.readLine().trim());

        int firstWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> firstWatch = IntStream.range(0, firstWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
```

```
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int repeatWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> repeatWatch = IntStream.range(0, repeatWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.findMinMinutes(m, firstWatch, repeatWatch);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 2	Easy	Sample	Success	1	0.0472 sec	33.8 KB
Testcase 0	Easy	Sample	Success	1	0.0507 sec	33.7 KB
Testcase 1	Easy	Sample	Success	1	0.0563 sec	33.6 KB

Testcase 3	Easy	Hidden	Success	4	0.1538 sec	33.6 KB
Testcase 4	Easy	Hidden	Success	4	0.0652 sec	34.5 KB
Testcase 5	Easy	Hidden	Success	4	0.1262 sec	42.2 KB
Testcase 6	Medium	Hidden	Success	5	0.4174 sec	67.8 KB
Testcase 7	Medium	Hidden	Success	5	0.6299 sec	164 KB
Testcase 8	Medium	Hidden	Success	5	0.5809 sec	144 KB
Testcase 9	Medium	Hidden	Success	5	0.553 sec	145 KB
Testcase 10	Medium	Hidden	Success	5	0.6871 sec	144 KB
Testcase 11	Medium	Hidden	Success	5	0.4597 sec	140 KB
Testcase 12	Hard	Hidden	Success	10	0.2747 sec	62.5 KB
Testcase 13	Hard	Hidden	Success	10	0.5537 sec	111 KB
Testcase 14	Hard	Hidden	Success	10	0.0503 sec	33.7 KB

No comments.